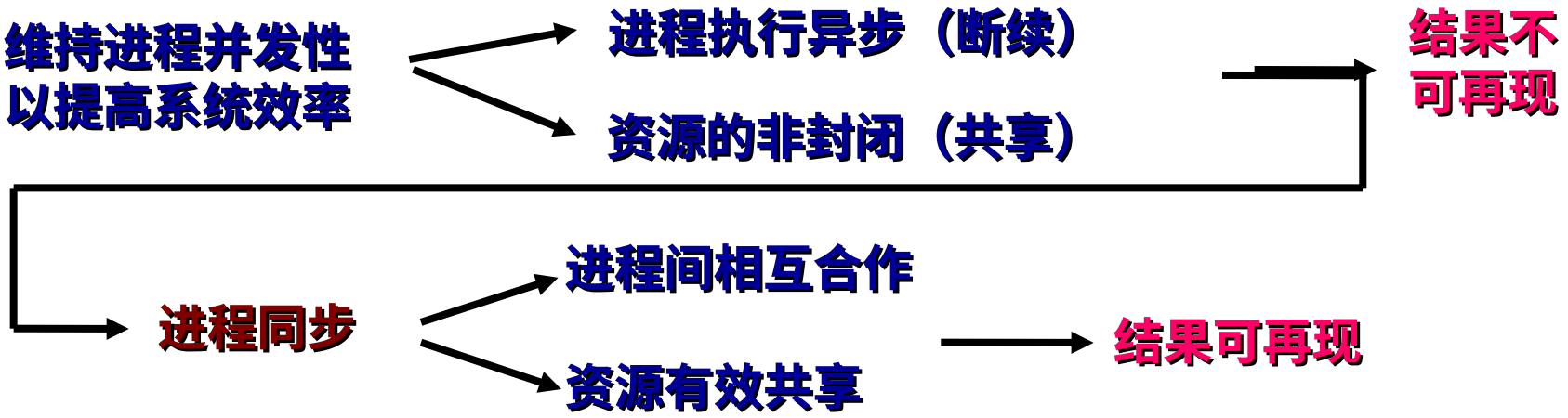
第四章 进程同步

- 进程同步问题的提出
  - 进程异步推进可能造成混乱
  - 混乱可能导致不可再现
- 进程同步目标



4.1 进程同步的基本概念

1．进程间两种主要关系

进程间的关系是在进程间相对独立的前提下发展的。

- **独立获得资源**
- **独立调度**

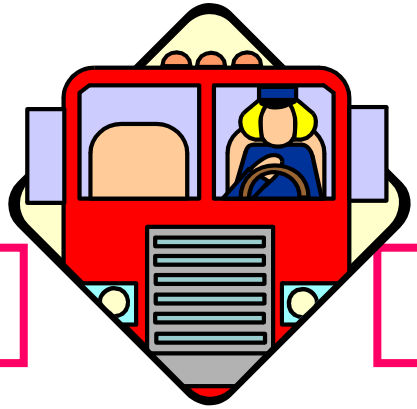
1) 直接作用 --- 相互合作引起 --- 进程同步：（如：输入、计算、打印）

进程间的相互联系是有意识的安排的，直接作用只发生在相交进程间。

2) 间接作用 --- 资源共享引起 --- 进程互斥：（如：打印机资源）

进程间要通过某种中介发生联系，是无意识安排的，可发生在相交进程之间，也可发生在无关进程之间。

进程间的同步关系（一）



司机

售票员

正常行车

售票

合作

到站停车

开车门

检查车况

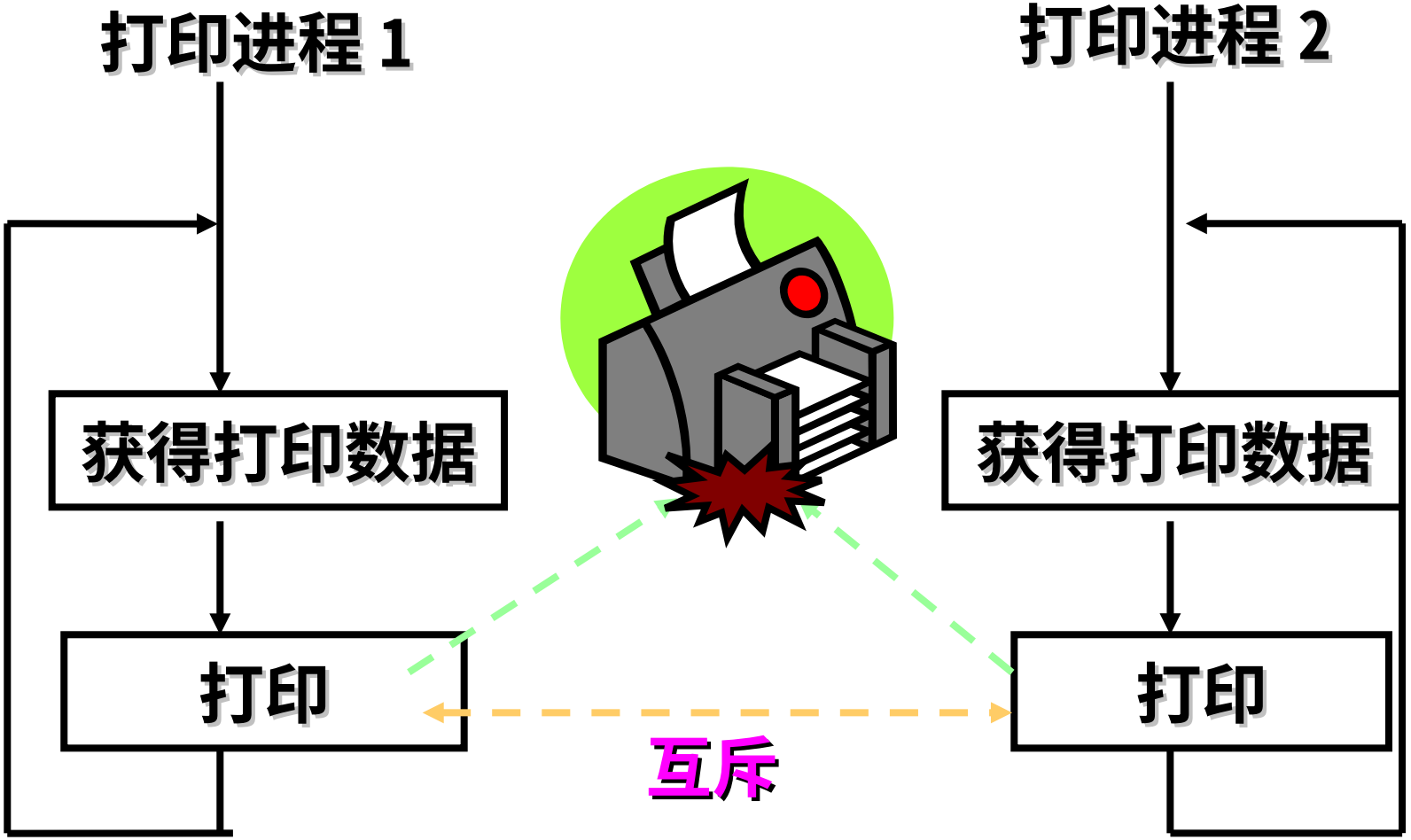
维持秩序

合作

开车

关车门

进程间的同步关系（二）



进程间的同步关系（三）

计算进程

完成数据计算

计算结果送到 Buffer

通知打印进程打印



Buffer

互斥

合作



打印进程

通知计算进程送下一个数

从 Buffer 中取数

打印

进程间的同步关系

相互合作 ←— 司机与售票员

竞争资源 ←— 计算者与打印者

竞争资源 ←— 多个打印者

2. 进程的同步（直接作用）

指系统中多个进程中发生的事件存在某种时序关系，需要相互合作，共同完成一项任务。具体说，一个进程运行到某一点时要求另一伙伴进程为它提供消息，在未获得消息之前，该进程处于等待（阻塞）状态，获得消息后被唤醒进入就绪状态。多个相关进程在执行次序上的协调。

如：司机　售票员

停车 ---→ 开车门

开车← --- 关车门

3. 进程的互斥（间接作用）

由于各进程要求共享资源，而有些资源需要互斥使用，因此各进程间竞争使用这些资源，进程的这种关系为进程的互斥。

如：多个打印者竞争打印机；

4、临界资源

系统中某些资源一次只允许一个进程使用，称这样的资源为临界资源或互斥资源。

1）资源状态为临界： 0 或 1

2）对临界资源的访问必须是互斥。

3）许多硬件资源如打印机、磁带机是属于临界资源。

思考：除此之外，软件中的共享变量是否是临界资源呢?

生产者 - 消费者 (producer-consumer) 著名的进程同步问题。

问题描述：有一群生产者进程在生产产品，并将这些产品提供给消费者进程去消费。为使生产者进程与消费者进程能并发执行，在两者之间设置了一个具有 n 个缓冲区的缓冲池（循环缓冲），生产者进程将它所生产的产品放入一个缓冲区中；消费者进程可从一个缓冲区中取走产品去消费。

数据定义：

1） item buffer[n]

用一个数组表示具有 n 个 (0 ， 1 ，…， n-1) 缓冲区的缓冲池。

2）输入指针 in:

下一个可投放产品的缓冲区。 in:= (in+1) % n ；

3）输出指针 out：

下一个可从中获取产品的缓冲区。 out:= (out+1) % n 。

4）整型变量 counter：生产者和消费者两进程共享。

初始值为 0;

生产者进程向缓冲池中投放一个产品后，使 counter 加 1；

消费者进程从中取走一个产品时，使 counter 减 1；

5）局部变量 nextp：

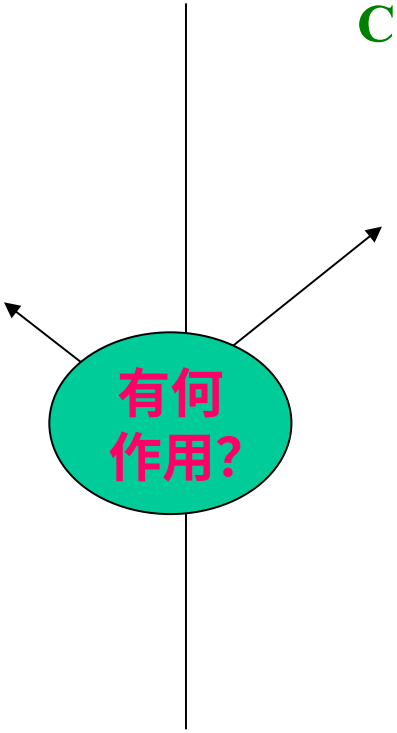
存放每次刚生产出来的产品;

6）局部变量 nextc：

存放每次要消费的产品。

Producer： 生产者进程

```
while (1){
produce an item in nextp ;
while (counter==n)  ;
buffer  [ in ]  :=nextp ;
in:=(in+1) % n ;
counter++ ;
} ;
```



Consumer: 消费者进程

```
while (1) {
while (counter==0)  ;
nextc:=buffer  [ out ]  ;
out:=(out+1)  % n ;
counter-- ;
consumer the item in nextc ;
} ;
```

初始 counter=0；

思考：

1、生产者程序和消费者程序顺序执行时其结果是否正确?

2、生产者程序和消费者程序并发执行时其结果是否正确?

3、原因何在?

问题：共享变量 counter 。生产者、消费者对其操作，这两个操作在用机器语言实现时，用下面的形式描述：假设：counter:=5

register1:=counter ；	register2:=counter ；
生产者： register1:=register1+1 ；	**消费者：** register2:=register2-1 ；
counter:=register1 ；	counter:=register2 ；

执行顺序：

1 、生产者先执行左列的三条语句，消费者再执行右列三条语句。 count:= 5

2 、消费者先执行右列的三条语句，生产者再执行左列三条语句。 count:= 5

3 、

register1:=counter ；	**(register1=5)**
register1:=register1+1 ；	**(register1=6)**
register2:=counter ；	**(register2=5)**
register2:=register2-1 ；	**(register2=4)**
counter:=register1 ；	**(counter=6)**
counter:=register2 ；	**(coun 结果不可再现**

结论：共享变量 counter 是临界资源，需互斥访问。

例题：

1. 有两个并发执行的进程 P1 和 P2，共享初值为 1 的变量 x。P1 对 x 加 1，P2 对 x 减 1。加 1 和减 1 操作的指令序列分别如下所示。

```
// 加 1 操作
load R1， x // 取 x 到寄存器 R1 中
inc R1
store x， R1 // 将 R1 的内容存入 x
```

```
// 减 1 操作
load R2， x
dec R2
store x， R2
```

两个操作各执行一次结束后，x 的值 (　　)。

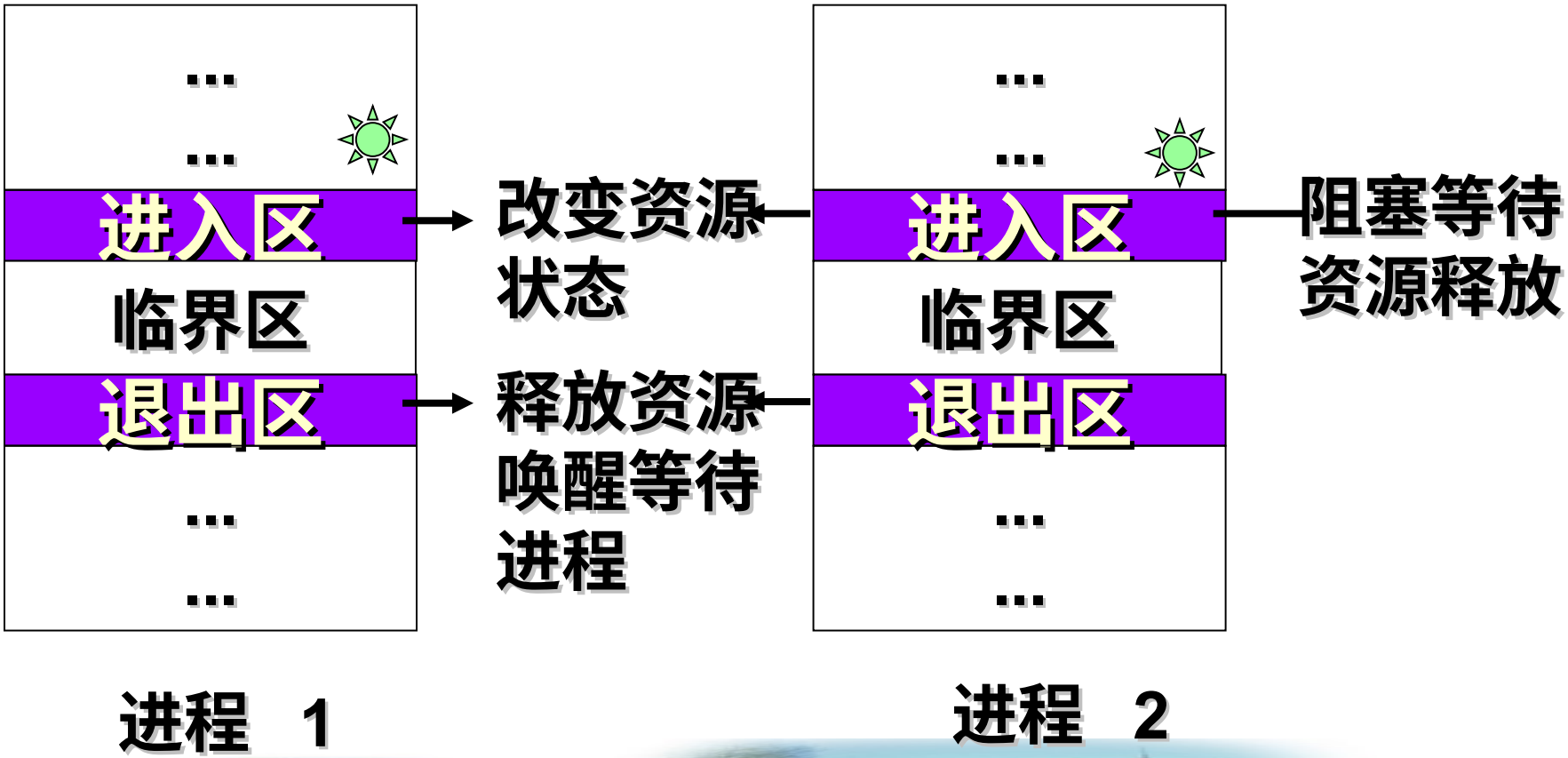
A．可能为 -1 或 3　　B．只能为 1

C．可能为 0、1 或 2　　D．可能为 -1、0、1 或 2

5. 临界区

每个进程用于访问临界资源的那段程序。

若能保证诸进程互斥地进入自己的临界区，便可实现诸进程对临界资源的互斥访问。



6. 进程同步应遵循的原则

1）空闲让进

当资源空闲时，应当允许访问资源的进程进入临界区。

2）忙则等待

当资源被占用时，应使申请访问该资源的进程等待，等待使用者归还资源。

3）让权等待

在进程等待资源时，从执行态转为阻塞态，应当让出 CPU 的使用权。系统将把 CPU 分配给其它进程使用，以提高系统效率。

4）有限等待

系统应保证等待的进程能在有限的时间内获得资源，继续执行，以防止无限等待浪费该进程已占用的资源。

4.4 信号量机制（semaphore）

– 经典信号量、记录型信号量、 AND 信号量、信号量集

1. 信号量机制的基本概念

– 信号量

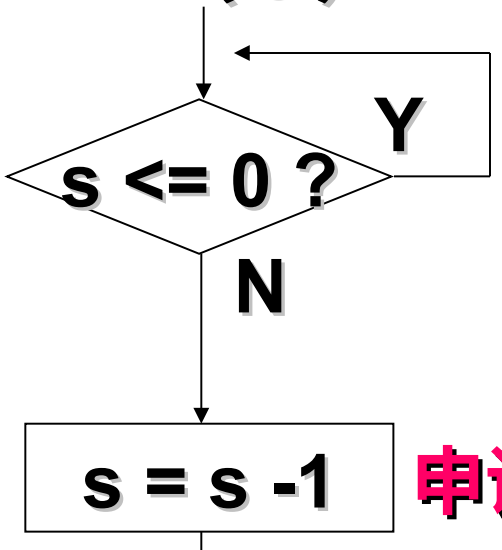
- 信号量是对具体物理资源的抽象。
- 不同类的资源用不同名称的信号量代表。
- 同类资源的个数用 > 0 的信号量值表示。
- 信号量值为 0 或 1 的信号量表示临界资源。

信号量是比锁更高级的资源抽象方式

2. 经典 (整型) 信号量的 P ， V 操作

– 资源的申请与释放－原语

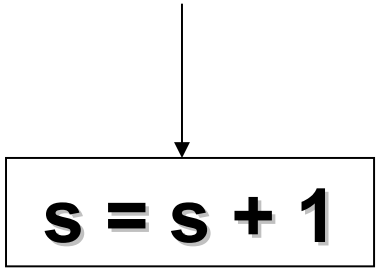
- P 操作　wait(S) ｛　while　S<=0 ；　//do no-op
- 　　S-- ；　申请一个资源；
- V 操作　signal(S) { S++;)　释放一个资源；



信号量：

忙等?

申请一个资源



释放一个资源

临界区 / 资源访问区

注意： P,V 两个原子操作执行时是不可中断的。

3. 记录型信号量

- 解决经典信号量机制中未遵循“让权等待”。
- 引入进程阻塞机制
- 在信号量里增加对阻塞进程的纪录。

```
typedef  struct {
   int value ;
   struct process_control_block *list ;
                    } semaphor;
```

资源个数

阻塞进程（PCB）队列

```
wait(semaphore *S)  // 申请资源  P(S)
{
  S->value-- ;
  if S->value<0 then   block(S->list) ;
};
```

```
signal(semaphore  *S) // 释放资源  V(S)
{
  S->value:++ ;
  if S->value<=0 then wakeup(S->list) ;
};
```

纪录型信号量机制特点：

– 进程对资源访问的过程：



– 原语保证

- p （s），v （s）操作都是原语；
- 保证不出现“锁不住”资源的现象；

例题：

1. 有一个计数信号量 S；

1）假如若干个进程对 S 进行了 28 次 P 操作和 18 次 V 操作之后，信号量 S 的值为 0；

2）假如若干个进程对信号量 S 进行了 15 次 P 操作和 2 次 V 操作。请问此时有多少个进程等待在信号量 S 的队列中（）。

A. 2　　B. 3　　C. 5　　D. 7

2. 设与某资源关联的信号量（K）初值为 3，当前值为 1。若 M 表示该资源的可用个数，N 表示等待该资源的进程数，则 M，N 分别是（）。

A. 0，1　　B. 1，0　　C. 1，2　　D. 2，0

3. 在 9 个生产者、6 个消费者共享容量为 8 的缓冲器的生产者 - 消费者问题中，互斥使用缓冲器的信号量初始值为（）。

A. 1　　B. 6　　C. 8　　D. 9

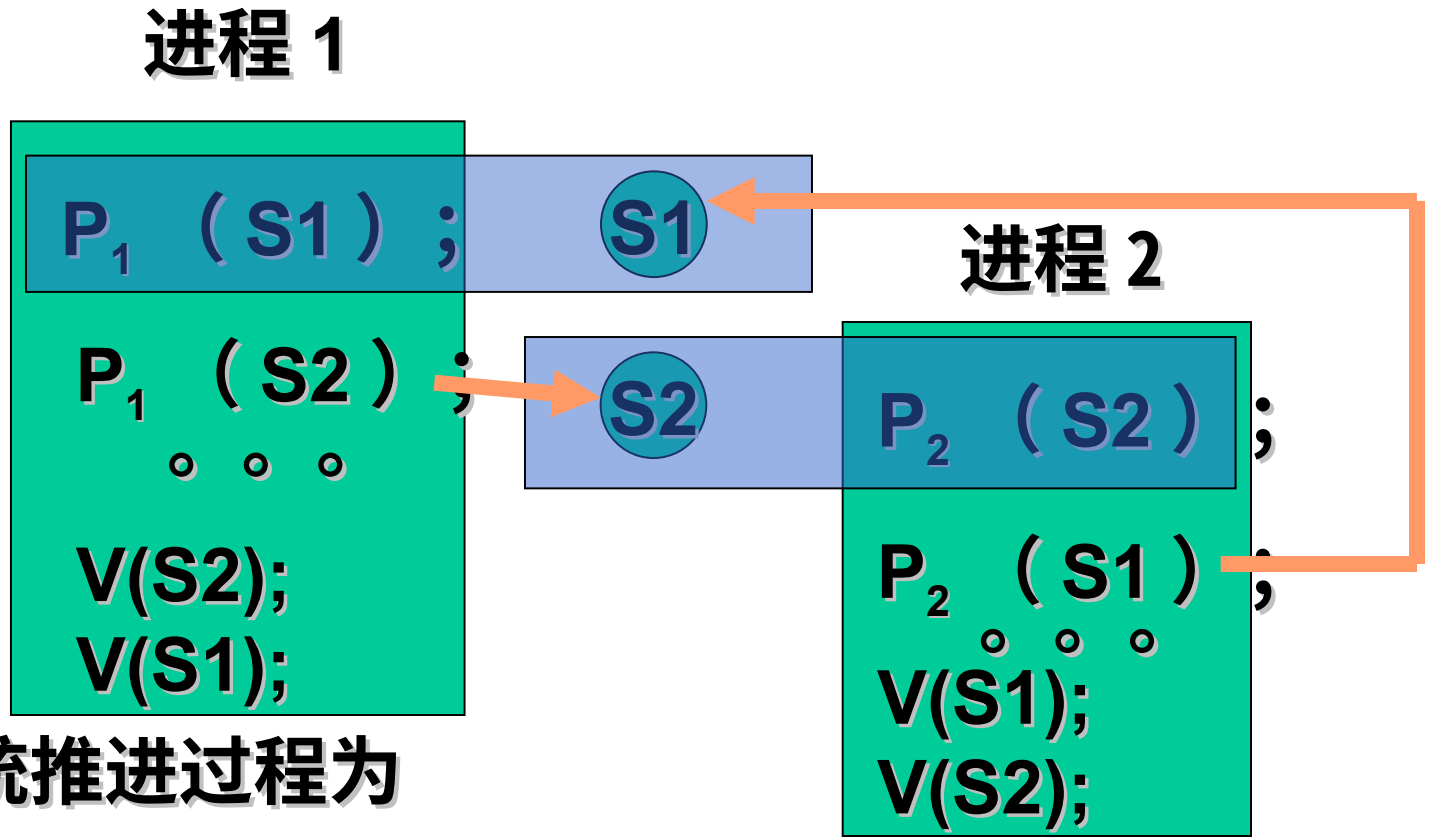
4. 有三个进程共享同一程序段，而每次只允许两个进程进入该程序段，若用 PV 操作同步机制，则信号 S 的取值范围是（）。

A. 2，1，0，-1　　B. 3，2，1，0　　C. 2，1，0，-1，-2　　D. 1，0，-1，-2

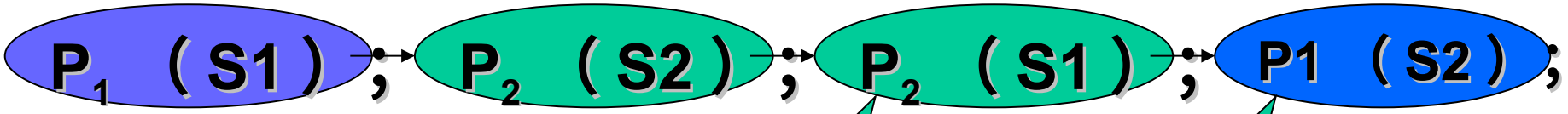
4. AND 型信号量

- 引入原因
- 基本思想
- 流程

AND 型信号量引入原因



系统推进过程为



进程 1 和进程 2 都无法继续推进出现死锁

进程 2 阻塞

进程 1 阻塞

AND 型信号量基本思想

基本思想

– 将多次对多个信号量的申请改为一次，用一个原子操作完成。

—— And 型信号量

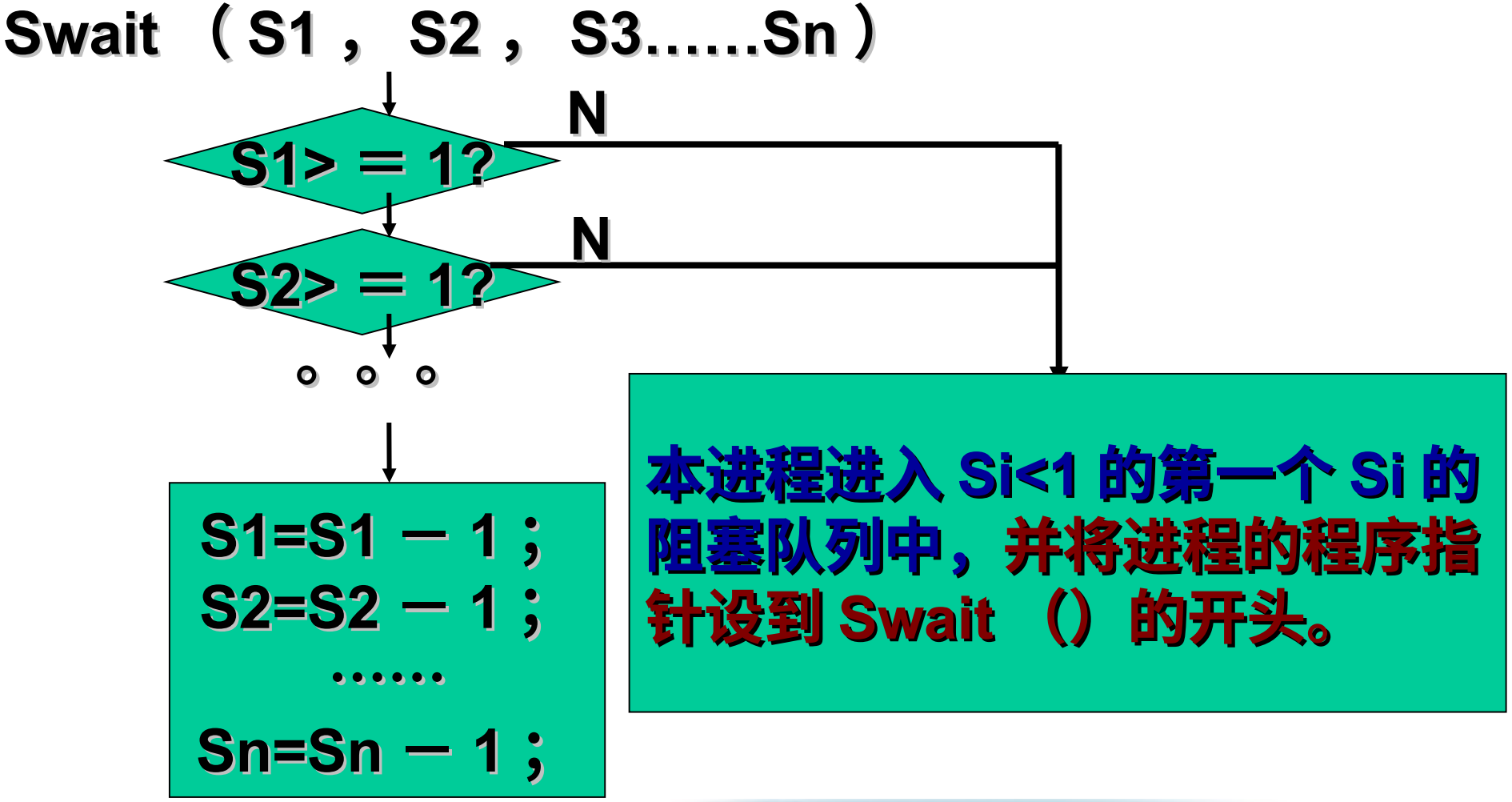
Swait （S1 ， S2 ， S3……Sn ）

或

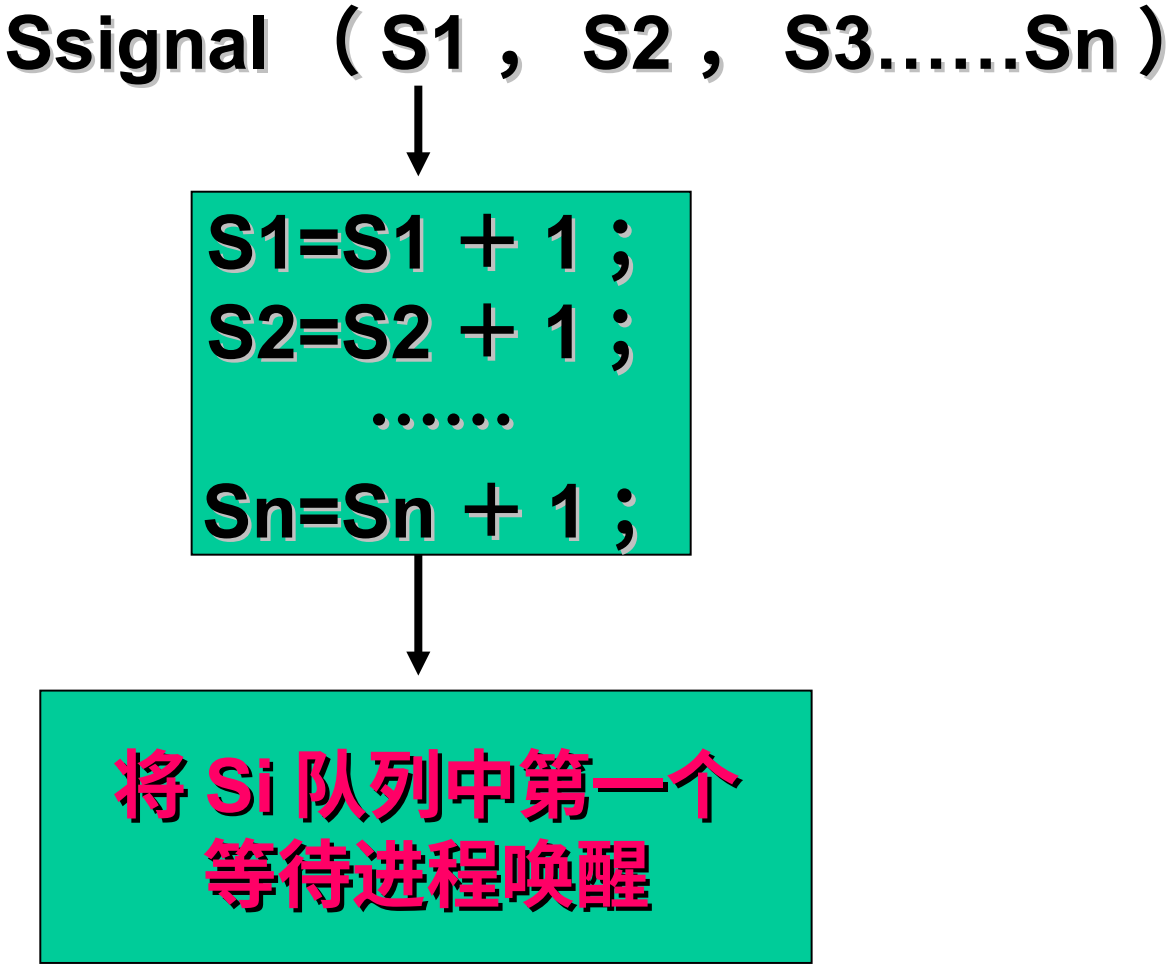
SP （S1 ， S2 ， S3……Sn ）

– 进程要么一次获得所有的资源，要么一个也申请不到。
  • 不会存在互相等待的局面

AND 型信号量集流程－－资源申请



AND 型信号量集流程－－资源释放



信号量集流程

进程 1

P_1（S1）；
$Swait_1$（S1,S2）
P_1（S2）；

进程 2

P_2（S2）；
$Swait_2$（S1,S2）
P_2（S1）；

系统推进

$Swait_1$（S1,S2）→ $Swait_2$（S1,S2）

或

$Swait_2$（S1,S2）→ $Swait_1$（S1,S2）

5. 信号量集

Swait （S1,t1,d1,……,Sn,tn,dn)

- 引入原因
  - 更灵活
- 基本思想
  - si： 各信号量；
  - ti： 申请下限
    - ti > 0 时，可进行资源预留；
  - di： 申请个数
    - 一次可申请一种资源的多个。

“信号量集”的几种特殊情况：

(1) Swait(S ， d ， d)。此时在信号量集中只有一个信号量 S ，但允许它每次申请 d 个资源，当现有资源数少于 d 时，不予分配。

(2) Swait(S ， 1 ， 1)。此时的信号量集已蜕化为一般的记录型信号量 (S>1 时) 或互斥信号量 (S=1 时)。

(3) Swait(S ， 1 ， 0)。这是一种很特殊且很有用的信号量操作。当 S≥1 时，允许多个进程进入某特定区；当 S 变为 0 后，将阻止任何进程进入特定区。换言之，它相当于一个可控开关。

S>=1 ， S 值不变，可以进入临界区；

S<1 ，阻塞进程；

小结四种信号量特点：

P V 操作

1、整型信号量 wait(s)， signal(s)— 对一类资源申请和释放

wait(s)：如果 s<=0，忙等；否则申请一个资源；

signal(s)：释放一个资源；

2、记录型信号量 -- 对一类资源申请和释放

解决了忙等；

wait(s)： 如 s 减一后， s<0 ，则阻塞，排到阻塞队列。

signal(s)：如 s 加一后， s<=0 ，则唤醒一个进程；

如 s=-2 ，说明有两个进程排在等待该类资源的阻塞队列。

3、 And 型信号量

解决了多类资源同步和互斥管理。策略：一次性全获得和释放；

但一次只能申请和释放一个资源。

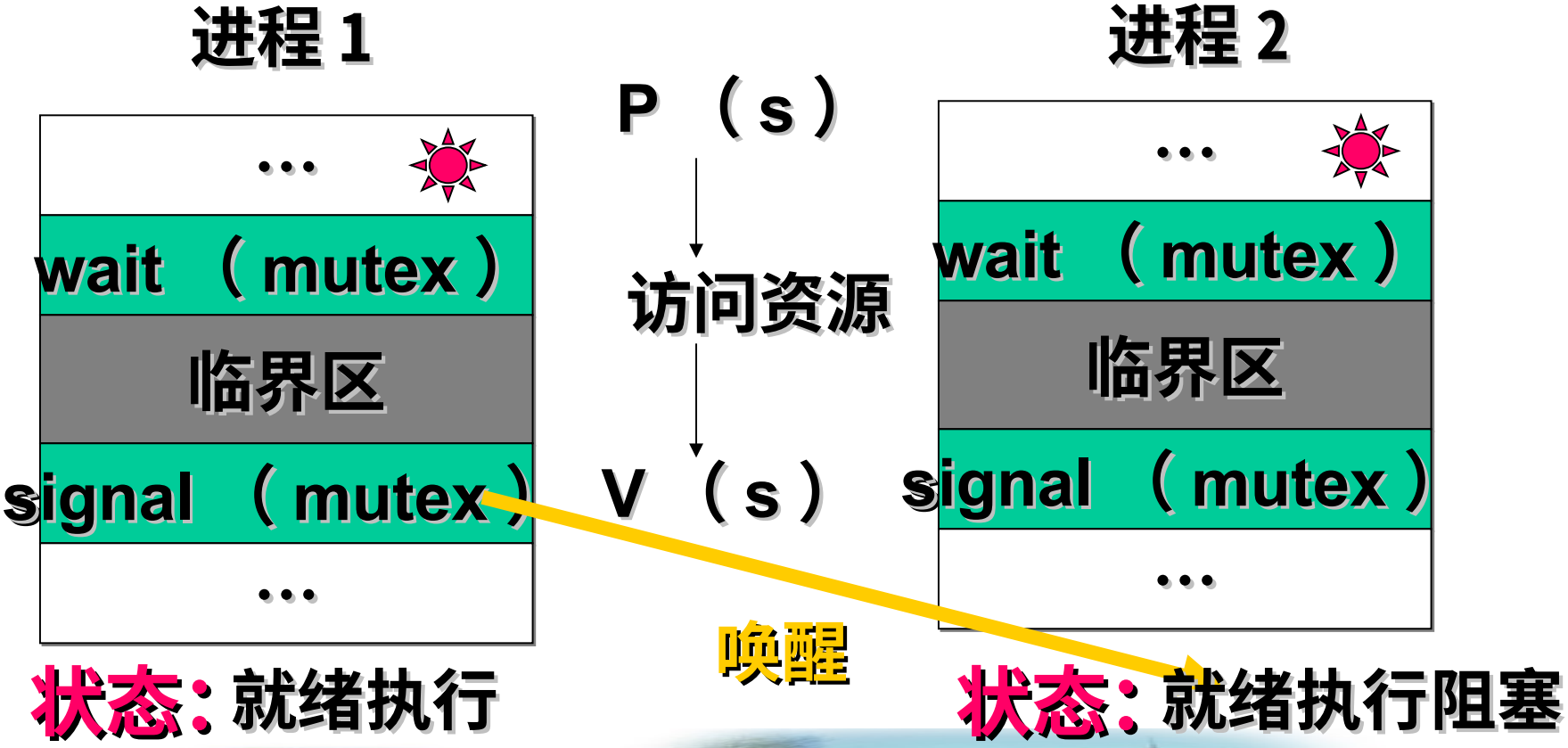
4、信号量集

对多类资源，每次可申请和释放多个，并且有一个下限。策略：如满足条件，一次性全获得和释放。

6．信号量的应用－－利用信号量实现进程互斥

－资源竞争时的进程同步。

- 对竞争资源的互斥访问－互斥信号量 mutex=1 。互斥信号量初值一般为 1 。

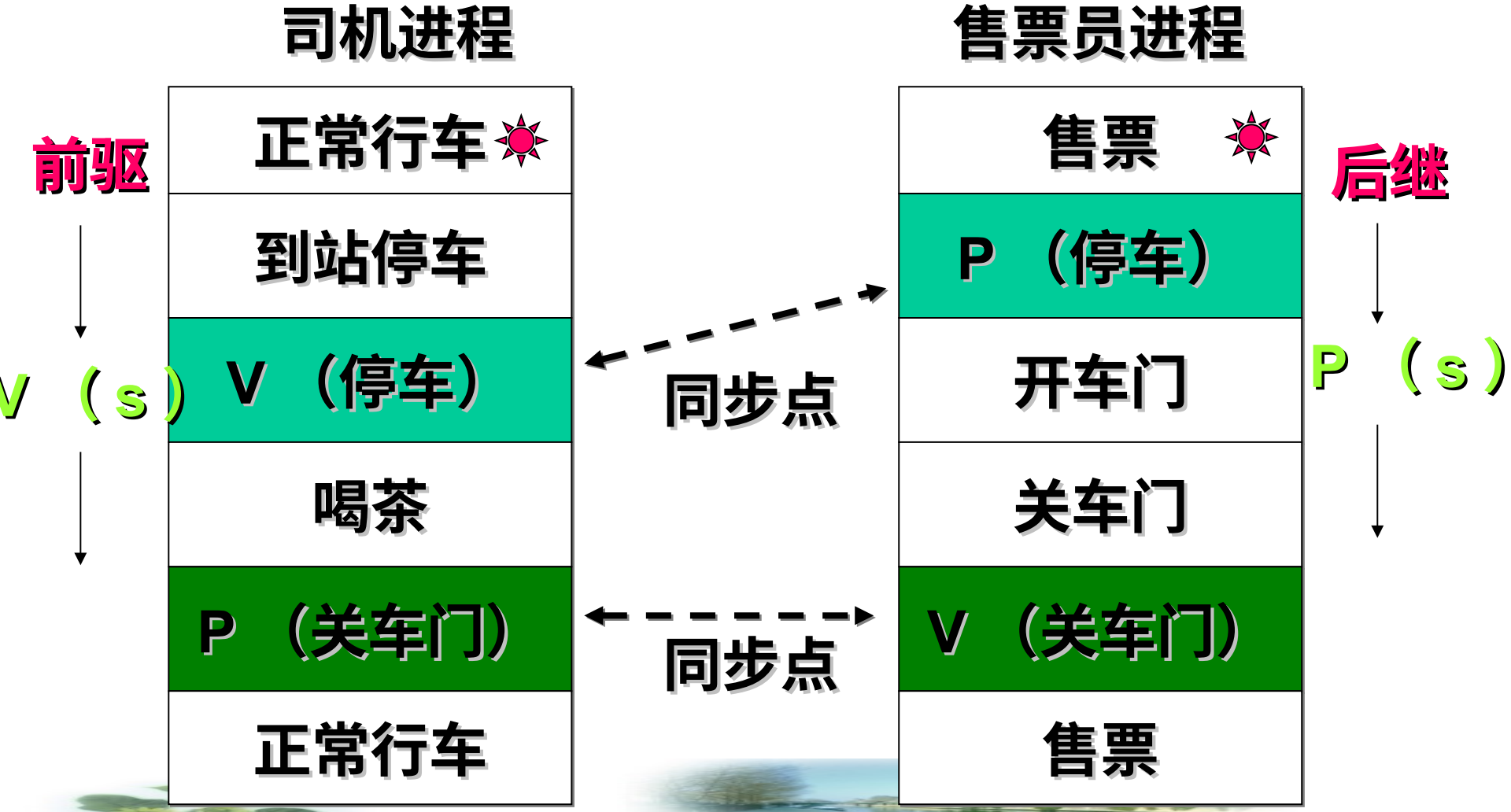


6. 信号量的应用 --- 利用信号量实现进程同步

-- 相互合作时的进程同步

- 保证进程间的前驱、后继关系

信号量初值为 0



6．信号量的应用 --- 利用信号量实现前趋关系

条件：设有两个并发执行的进程 P_1 和 P_2。 P_1 中有语句 S_1； P_2 中有语句 S_2。

目标： $S_1 \longrightarrow S_2$ 前驱关系

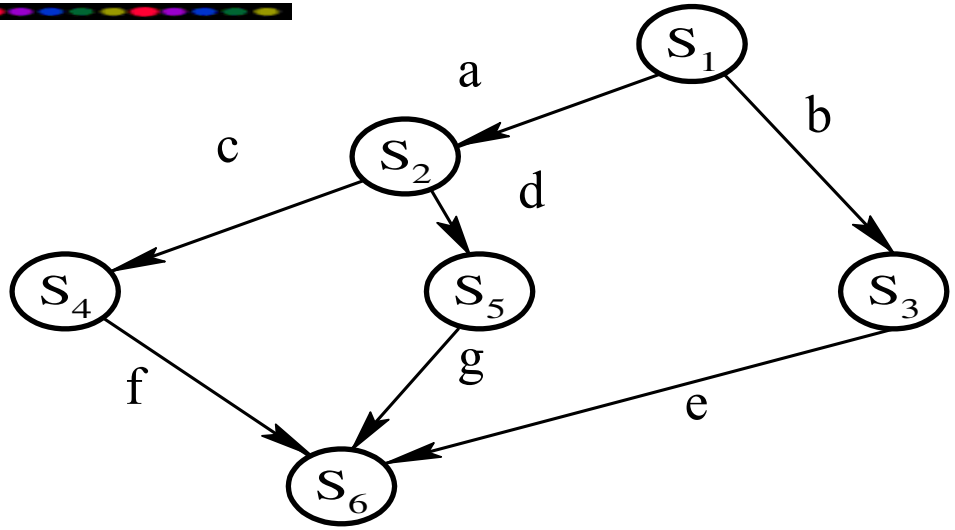
实现： P1 和 P2 共享一个公用信号量，初值 S=0;

P_1 : S_1； signal(S)；

P_2 : wait(S)； S_2；

例：

有一前驱图，利用信号量实现前驱关系。




```
Var a,b,c,d,e,f,g： semaphore: =0,0,0,0,0,0,0 ;
      begin
        parbegin
          begin S1 ;  signal(a) ;  signal(b) ;  end ;
          begin wait(a) ;  S2 ;  signal(c) ;  signal(d) ;  end ;
          begin wait(b) ;  S3 ;  signal(e) ;  end ;
          begin wait(c) ;  S4 ;  signal(f) ;  end ;
          begin wait(d) ;  S5 ;  signal(g) ;  end ;
          begin wait(e) ;  wait(f) ;  wait(g) ;  S6 ;  end ;
        parend
```

7. 利用信号量实现同步（X→Y）

```
Semaphore s=0;    // 初始化信号量
P1() {
        …
         x;              // 语句 x
         V(s) ;          // 告诉进程 P2 ，语句 x 已经完成
        …
}

P2() {
        …
        P(s) ;          // 检查语句 x 是否运行完成
        y;              // 检查无误，运行语句 y
        …
}
```

7. 利用信号量实现互斥

```
Semaphore s=1;   // 初始化信号量
P1() {
        …
        P(s)；              // 准备开始访问临界资源，加锁
        进程 P1 的临界区；
        V(s)；              // 访问结束，解锁
        …
}
P2() {
        …
        P(s)；              // 准备开始访问临界资源，加锁
        进程 P2 的临界区；
        V(s)；              // 访问结束，解锁
        …
}
```

2.6　经典进程的同步问题

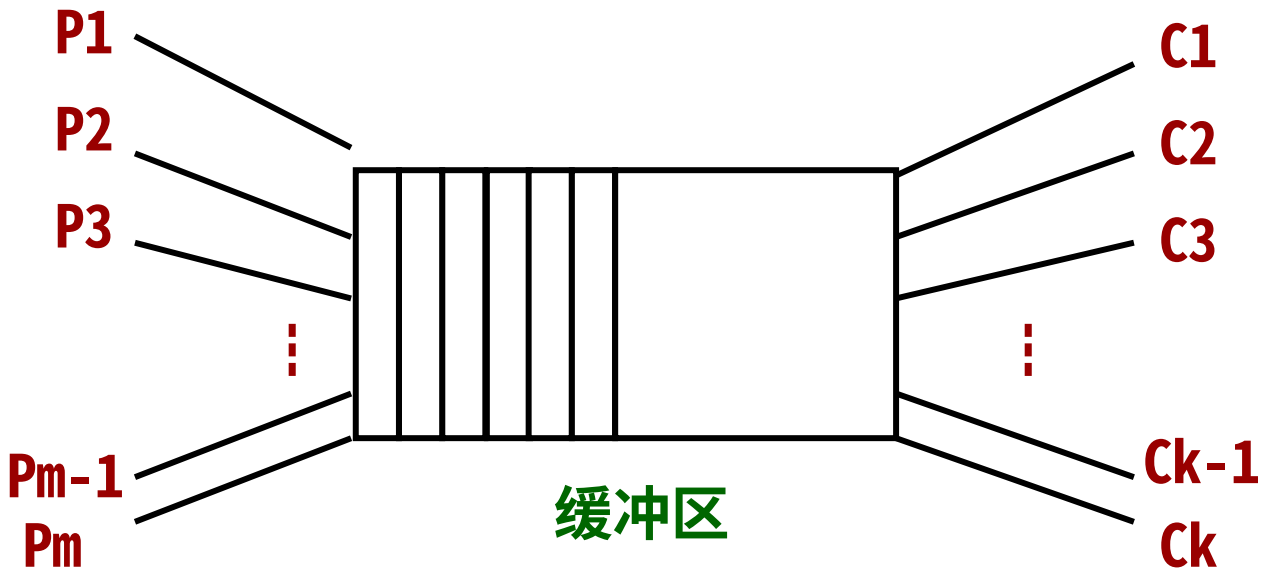
一、生产者—消费者问题 --- 典型的同步例子

它描述了一组生产者向一组消费者提供产品（数据），它们共享一个有界缓冲区，生产者向其中投放产品，消费者从中取出产品消费。这样的一个问题是许多相互合作进程存在的内在关系的一种抽象。

一组生产者 P1 、 P2 、…、 Pm

一组消费者 C1 、 C2 、…、 Ck

有界缓冲区——长度为 n （ n ＞ 0 ）



只要缓冲区未满，生产者就可把产品送入缓冲区；只要缓冲区未空，消费者便可从缓冲区取走产品并消耗它。

仅当缓冲区满时，生产者被阻塞，类似地，缓冲区空时，消费者被阻塞。

生产者和消费者的同步关系将禁止生产者向满的缓冲区输送产品，也禁止消费者从空的缓冲区中提取物品。

算法分析

两类进程：生产者进程和消费者进程

– （1）进程间的关系—（合作：同步信号量）

- 生产者生产产品后消费者消费。设 full: 表示满缓冲区 (即产品）的数目，初值为 0 ；

消费者（后继）：　→　P （full）　→

生产者（前驱）：　→　V （full）　→

消费者何时阻塞?

- 消费者消费后的空缓冲区由生产者生产产品。设 empty: 表示空缓冲区的数目，初值为缓冲区的大小 n 。

生产者（后继）：　→　P （empty）　→

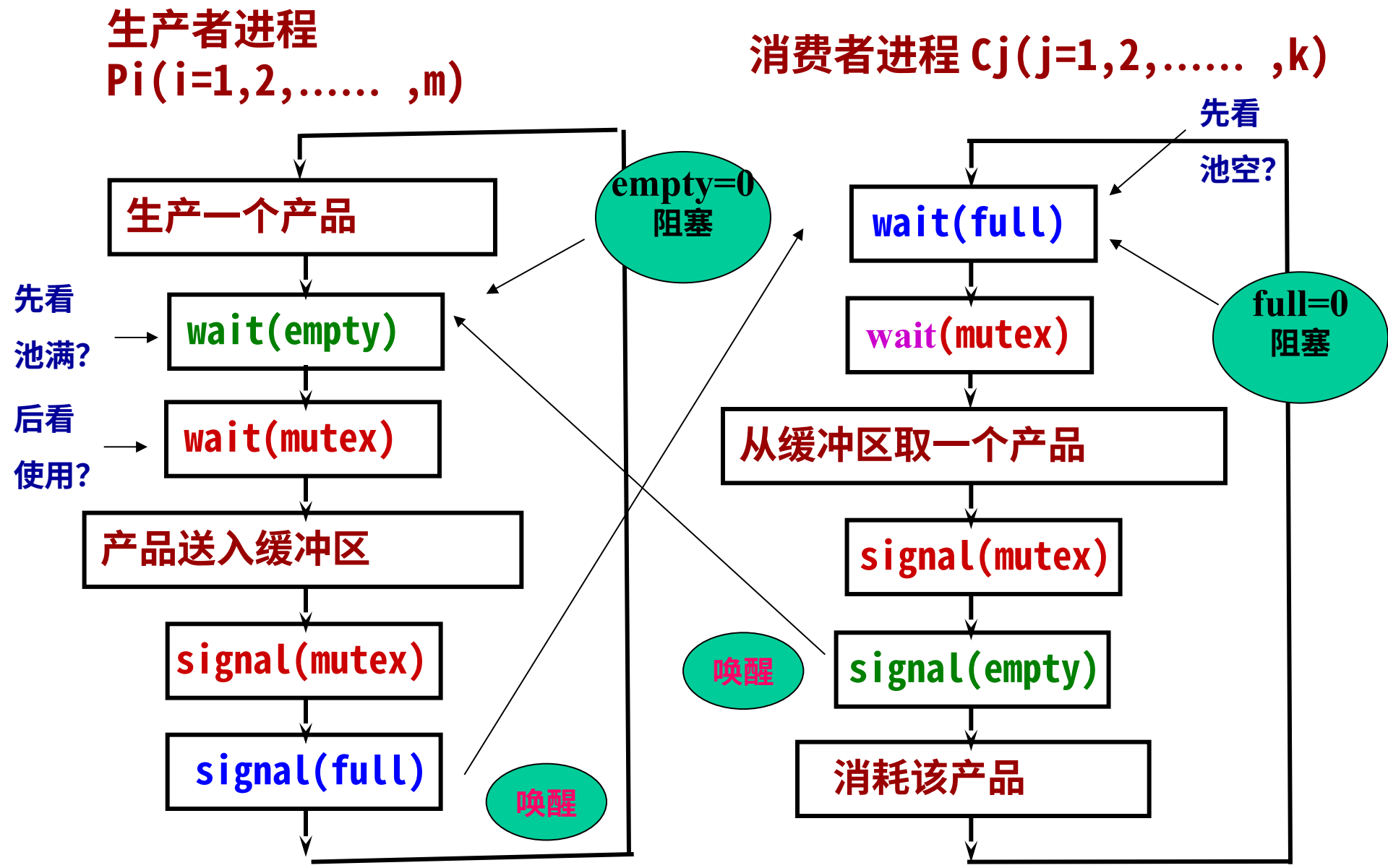
消费者（前驱）：　→　V （empty）　→

生产者何时阻塞?

– （2）进程间的关系—（竞争：互斥信号量）

由于有界缓冲区是一个临界资源，必须互斥使用，所以，还需要设置一个互斥信号量：　mutex: 互斥信号量，初值为 l 。

生产者—消费者问题流程


生产者进程
Pi(i=1,2,..... ,m)
消费者进程 Cj(j=1,2,..... ,k)
生产一个产品
wait(empty)
wait(mutex)
产品送入缓冲区
signal(mutex)
signal(full)
先看
池满?
后看
使用?
empty=0
阻塞
唤醒
先看
池空?
wait(full)
wait(mutex)
从缓冲区取一个产品
signal(mutex)
signal(empty)
消耗该产品
full=0
阻塞
唤醒

Semaphore mutex=1, empty=n, full=0；

item buffer[n];

int in=0， out=0；

```
void proceducer()
  {
     do{
        producer an item  nextp；
        Swait(empty,mutex)
        buffer(in)=nextp；
        in=(in+1) % n；
        Ssignal(mutex,full)
     } while (TRUE);
  }
```

```
void consumer()
 {
    do{
       Swait(full,mutex)
       nextc=buffer(out)；
       out=(out+1) % n；
       Ssignal(mutex,empty)
       consumer the item in nextc；
    }while(TRUE);
 }
```

And 型信号量

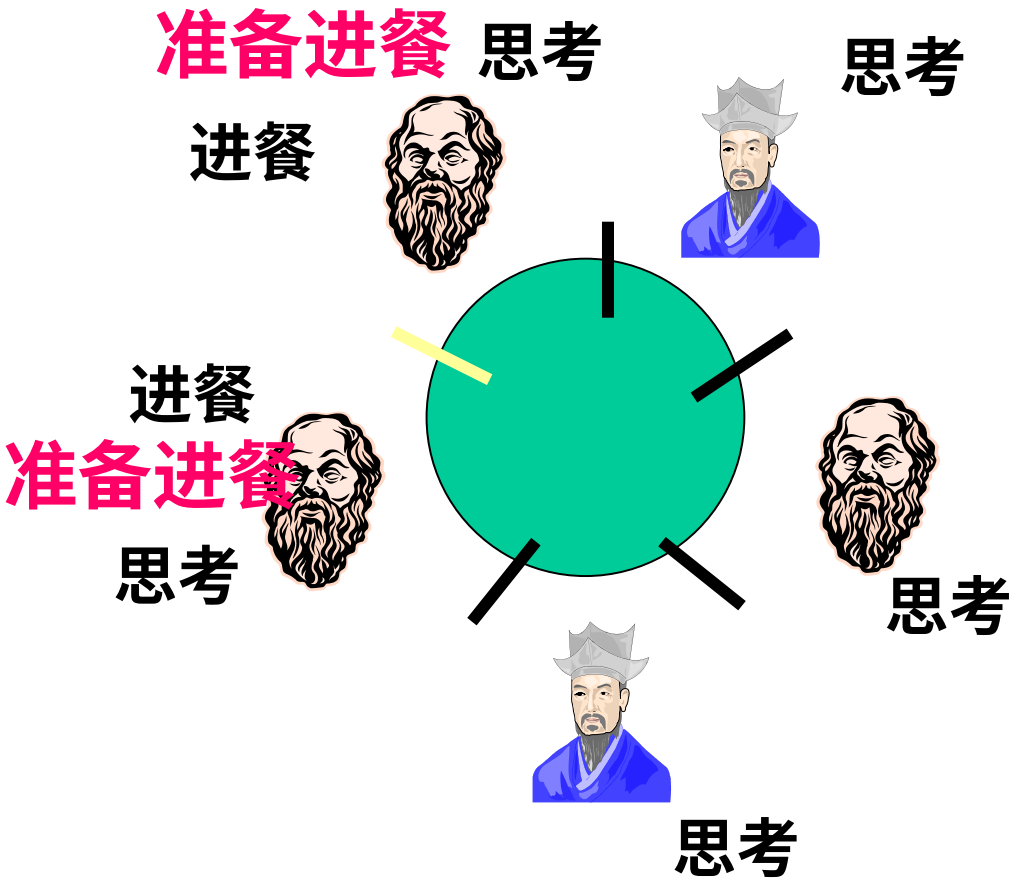
生产者消费者算法小结：

- 在分析进程同步问题中，逐个分析进程间的关系是关键。
- 不管多复杂的关系，总能归结为两种基本关系（竞争与合作），总是这两种关系的组合。
- 不管互斥信号量还是同步信号量，信号量的使用必须成对出现。
- 互斥信号量初值设为 1；
- P(s1) ， P(s2) 两操作在一起， P 的顺序至关重要，同步 P 和互斥 P 在一起，同步 P 操作在前。
- 互斥操作时， PV 处于同一进程；同步操作时，不在同一进程

二、哲学家进餐问题

—问题描述

- 五位哲学家围桌而坐。
- 哲学家在思考问题时不需要任何资源，思考完问题后进入进餐态。
- 每人必须获得左右两支筷子才能进餐。



1．利用记录型信号量解决哲学家进餐问题

放在桌子上的筷子是临界资源，在一段时间内只允许一位哲学家使用。用一个信号量表示一只筷子，由这五个信号量构成信号量数组。

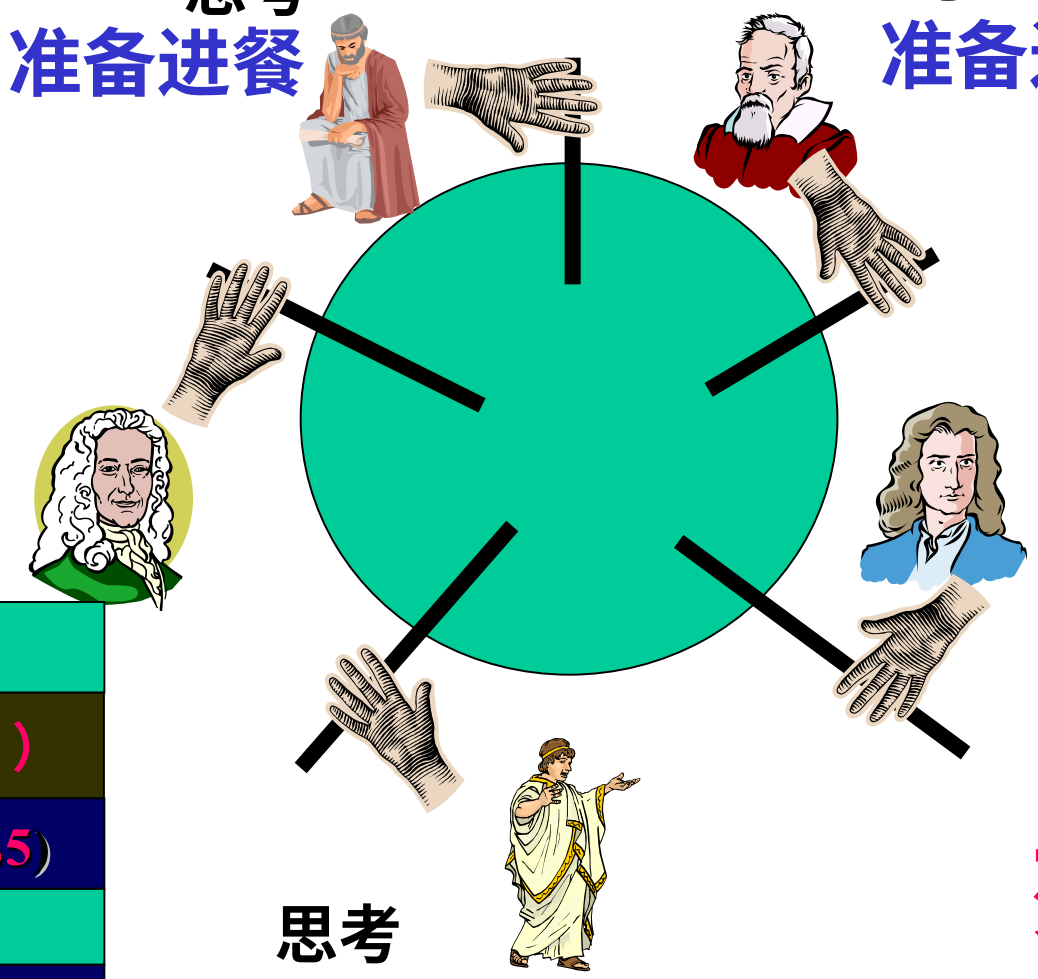
信号量初始值均为 1；

```
semaphore chopstick[5] ={1,1,1,1,1}；
```

活动描述：

```
do{
      wait(chopstick[i])；  // 拿起左边筷子
      wait(chopstick[(i+1) %5])；  // 拿起右边筷子
      eat；
      signal(chopstick[i])；  // 放下左边筷子
      signal(chopstick[(i+1) % 5])；  // 放下右边筷子
      think；
}while(TRUE)；
```

哲学家问题分析



思考

P （chopstick[i]）

P(chopstick[i+1] %5)

eat

V （chopstick[i]）)

V(chopstick[i+1] %5)

死锁

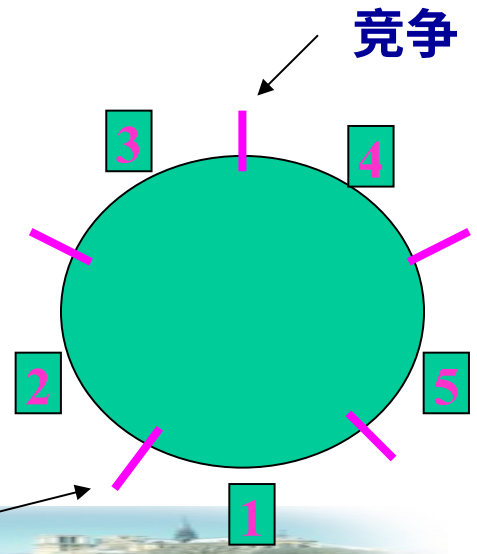
可采取以下几种解决死锁问题方法：

(1) 至多只允许有四位哲学家同时去拿左边的筷子，最终能保证至少有一位哲学家能够进餐，并在用毕时能释放出他用过的两只筷子，从而使更多的哲学家能够进餐。

(2) 仅当哲学家的左、右两只筷子均可用时，才允许他拿起筷子进餐。

(3) 规定奇数号哲学家先拿他左边的筷子，然后再去拿右边的筷子，而偶数号哲学家则相反。

注：上述几种方法可以解决死锁，但都施加了一些限制，对系统的效率会产生一定的影响。



2．利用 AND 信号量机制解决哲学家进餐问题

在哲学家进餐问题中，要求每个哲学家先获得两个临界资源 (筷子) 后方能进餐， AND 信号量机制可获得最简洁的解法。描述如下：

```
Semaphore chopstick[5]={1,1,1,1,1}；
        do{
            think；
            Swait(chopstick[(i+1) % 5]， chopstick[i])；
            eat；
            Ssignal(chopstick[(i+1) % 5]， chopstick[i])；
        }while(TRUE)；
```

三、读者—写者问题

（Reader and Writer ）

- 问题描述
  - 一个数据记录 (共享数据)，有多个进程进行读操作，另一些进程进行修改操作。
    - （Reader ）
    - （Writer ）
  - 读写策略
    - 允许多个进程同时进行读操作——读不互斥
    - 不允许多于一个进程进行写操作——写互斥
    - 不允许读写操作同时进行——读写互斥

用一个计数器 Readcount 来统计读者的数量，每当有一个读者欲访问共享数据时，对 Readcount 计数加 1 ，每当有一个读者结束对共享数据的访问时，对 Readcount 计数减 1 ，在第一个读者到达共享数据区之后以及最后一个读者离开共享数据区之前，写者不能访问共享数据区。

思考：需要几个信号量？是什么类型信号量?

为了解决读者—写者问题，我们设置了两个互斥信号量：

信号量 wmutex ，用于写者与写者之间、读者与写者、写者与读者之间对共享数据区操作的互斥，初值为 1 ；

信号量 rmutex ，用于多个读者对共享计数器 Readcount(临界资源) 操作的互斥，初值为 1 。

读者—写者问题可描述如下：

Semaphore rmutex=1 ， wmutex1, readcount=0；

读者： **void reader() {**

```
do
  wait(rmutex)；  /* 读进程要进入先修改 Readcount*/
  if (readcount==0) then wait(wmutex)；
 /* 如尚无读进程，读进程需执行 wait(wmutex) 。
进入 1 、 如 wmutex=1( 成功 ) ，说明无写进程在写，
     则 wmutex=0 （锁住写进程），读进程可进入读。
  2 、 如 wmutex=0 （不成功），说明有写进程在写，则读进程阻塞等待。 */
  readcount++；  /*  如成功进入，读者加一 */
  signal(rmutex)；  /* 读进程对 readcount 访问结束 */
  perform read operation；  /* 执行读操作 */
  wait(rmutex)；  /* 读进程退出要修改 Readcount*/
  readcount--；  /*  读者减一  */
退出 if (readcount==0) then signal(wmutex)；
 /* 如是最后一个读进程，则 wmutex=1 （释放写进程），允许写进程进行写 */
  signal(rmutex)；
} while(TRUE)
}
```

写者： **void writer() {**

```
    do{
      wait(wmutex)；  // 加锁
      perform write operation；
      signal(wmutex)；  // 解锁
    }while(TRUE)；
}
```

补充：读者—写者问题分析

1、前面的算法，**读进程是优先的**，当存在读进程时，写操作将被延迟，并且有一个读进程活跃，随后来的读进程将被允许访问文件。这样将导致写进程可能长时间等待，且存在写进程“饿死”的情况。

2、如何希望**写进程优先**，即当有读进程正在读共享文件时，有写进程请求访问，这时应禁止后续的读进程的请求，等待到已在共享文件的读进程执行完毕则立即让写进程执行，只有在无写进程执行的情况下才允许读进程再次运行。

3、如何修改前面的算法让“写优先”，请同学们课后完成。

例 1：某超级市场，可容纳 100 人同时购物。入口处备有篮子，每个购物者可持一只篮子入内购物。出口处结帐，并归还篮子（出、入口仅容一人通过）。请试用 P （ S ）、 V （ S ）操作以及信号量写出购物同步算法。

解：设信号量 S 其初始值为 100 ，信号量 mutex 其初始值为 1 。购物者进程设为 Pi ，其算法如下所示：

```
s,mutex： semaphore；
s=100； mutex=1;
process Pi （i=1，
begin
  repeat
    P(s);
    P(mutex)；
    进入口处，并取一只篮子;
    V(mutex);
    选购商品
    P(mutex)；
    结帐，并归还篮子;
    V(mutex);
  until false;
    V(s)；
end ;
```